Interpolation

- Interpolation
- Linear interpolation
- Nearest Neighbor Interpolation
- Polynomial interpolation
  - Lagrange Interpolation
  - Newton’s Interpolation

Newton's Divided Difference Interpolation

Newton's divided difference interpolation formula is an interpolation technique used when the interval difference is not same for all sequence of values.

Newton's Divided Difference Interpolation

Construct a polynomial that interpolates data points $(x_0, f(x_0)), (x_1, f(x_1)), \ldots, (x_n, f(x_n))$.

Newton's polynomial uses a form:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1})$$

Newton's Divided Difference Interpolation

$$f(x) \approx a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1})$$

At $x = x_0$: $a_0 = f(x_0)$.

At $x = x_1$:

$$a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

At $x = x_2$:

$$a_2 = \frac{\big(f(x_2) - f(x_1)\big)/(x_2 - x_1) - \big(f(x_1) - f(x_0)\big)/(x_1 - x_0)}{x_2 - x_0}$$

Newton's Divided Difference Interpolation

- Generally, Difference

$$f[x_i] = f(x_i),$$

$$f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i},$$

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Newton's Divided Difference Interpolation

- Newton’s Formula

$$P_n(x) = f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2] + \cdots$$

$$\cdot\cdot\,(x - x_{n-1})f[x_0, \ldots, x_n]$$

Newton's Divided Difference Interpolation

- Example

Compute $f(9.2)$ from the values shown in the following table.

x_j	$f_j = f(x_j)$
8.0	2.079442
9.0	2.197225
9.5	2.251292
11.0	2.397895

Example

To find $f[x_0,x_1]$, $f[x_0,x_1,x_2]$, $f[x_0,\ldots,x_n]$ in Newton's interpolation formula, we need to construct a **difference table** as follows:

x_j	$f_j = f(x_j)$	$f[x_j, x_{j+1}]$	$f[x_j, x_{j+1}, x_{j+2}]$	$f[x_j, \cdots, x_{j+3}]$
8.0	2.079442			
		0.117783		
9.0	2.197225		−0.006433	
		0.108134		0.000411
9.5	2.251292		−0.005200	
		0.097735		
11.0	2.397895			

The values in the table are calculated as follows:

First column:

$$f[x_j,x_{j+1}]=\frac{2.197225-2.079442}{9-8}=0.117783$$

Example

$$f[x_j, x_{j+1}] = \frac{2.251292 - 2.197225}{9.5 - 9} = 0.108134$$

$$f[x_j, x_{j+1}] = \frac{2.397895 - 2.251292}{11 - 9.5} = 0.097735$$

Second column:

$$f[x_j, x_{j+1}, x_{j+2}] = \frac{0.108134 - 0.117783}{9.5 - 8} = -0.006433$$

$$f[x_j, x_{j+1}, x_{j+2}] = \frac{0.097735 - 0.108134}{11 - 9} = -0.0052$$

Third column:

$$f[x_j, \ldots, x_{j+3}] = \frac{-0.0052 + 0.006433}{11 - 8} = 0.000411$$

Example

The values needed in Newton's interpolation formula are circled:

$$f(x) \approx p_3(x) = 2.079442 + 0.117783(x - 8.0) - 0.006433(x - 8.0)(x - 9.0) + 0.000411(x - 8.0)(x - 9.0)(x - 9.5).$$

At $x = 9.2$,

$$f(9.2) \approx 2.079442 + 0.141340 - 0.001544 - 0.000030 = 2.219208.$$

Newton's Divided Difference Interpolation

- Previously, we studied **Newton's Divided Difference Formula**, where data points were **not equally spaced**.
- In practice, however, many datasets are recorded at **equal intervals** (e.g., time series, experimental data, tabular values).
- For such cases, we use:
  - **Newton's Forward Difference Formula (Δ)** – interpolation near the **beginning** of the table.
  - **Newton's Backward Difference Formula (∇)** – interpolation near the **end** of the table.

Δ (Delta) → Forward difference symbol

∇ (Nabla) → Backward difference symbol

Newton's Forward & Backward Difference

When data points are **equally spaced**, Newton's **forward** and **backward difference interpolation formulas** provide efficient ways to approximate function values.

- **Forward Difference Formula** → Best used when the interpolation point x is **near the beginning** of the data table.
- **Backward Difference Formula** → Best used when the interpolation point x is **near the end** of the data table.

For equally spaced points:

$$x_0,\ x_1 = x_0 + h,\ x_2 = x_0 + 2h, \ldots$$

where h is the spacing.

Newton's Forward & Backward Difference

The **forward difference operator** Δ is defined as:

$$\Delta f(x_j) = f(x_{j+1}) - f(x_j)$$

Higher-order forward differences:

$$\Delta^2 f(x_j) = \Delta f(x_{j+1}) - \Delta f(x_j), \quad \Delta^3 f(x_j) = \Delta^2 f(x_{j+1}) - \Delta^2 f(x_j), \ \ldots$$

Newton's Forward & Backward Difference

In this case, Newton's Forward Difference Formula can be used to carry out an interpolation process. For a function $f(x)$, this formula can be expressed as:

$$f(x) = f_0 + r\,\Delta f_0 + \frac{r(r-1)}{2!}\Delta^2 f_0 + \ldots.. + \frac{r(r-1)\ldots\ldots(r-n-1)}{n!}\Delta^n f_0$$

$$r = \frac{x - x_j}{h}$$

Newton's Forward & Backward Difference

Example: Compute cosh (0.56) from Newton's forward difference formula and the four values in the following table.

j	x_j	$f_j = \cosh x_j$
0	0.5	1.127626
1	0.6	1.185465
2	0.7	1.255169
3	0.8	1.337435

Newton's Forward & Backward Difference

We compute the forward differences as shown in the table below.

j	x_j	$f_j = \cosh x_j$	Δf_j	$\Delta^2 f_j$	$\Delta^3 f_j$
0	0.5	1.127626			
			0.057839		
1	0.6	1.185465		0.011865	
			0.069704		0.000697
2	0.7	1.255169		0.012562	
			0.082266		
3	0.8	1.337435			

Newton's Forward & Backward Difference

The values we need are circled.

In the general formula, we have $r = (0.56\text{-}0.5)/0.1 = 0.6$. So that the general formula gives:

$$\cosh 0.56 \approx 1.127626 + 0.6 \cdot 0.057839 + \frac{0.6(-0.4)}{2} \cdot 0.011865 + \frac{0.6(-0.4)(-1.4)}{6} \cdot 0.000697$$

$$= 1.127626 + 0.034703 - 0.001424 + 0.000039$$

$$= 1.160944.$$

Newton's Forward & Backward Difference

The **backward difference operator** ∇ is defined as:

$$\nabla f(x_j) = f(x_j) - f(x_{j-1})$$

Higher-order backward differences:

$$\nabla^2 f(x_j) = \nabla f(x_j) - \nabla f(x_{j-1}), \quad \nabla^3 f(x_j) = \nabla^2 f(x_j) - \nabla^2 f(x_{j-1}), \ \ldots$$

Newton's Forward & Backward Difference

Therefore, ***the first backward difference of f at x_j is defined by :***

$$\nabla f_j = f_j - f_{j-1},$$

the *second backward difference* of f at x_j by

$$\nabla^2 f_j = \nabla f_j - \nabla f_{j-1},$$

and, continuing in this way, the ***k*th backward difference** of f at x_j by

$$\nabla^k f_j = \nabla^{k-1} f_j - \nabla^{k-1} f_{j-1} \qquad (k = 1, 2, \cdots).$$

A formula similar to the forward formula but involving backward differences is Newton's backward difference interpolation formula

$$f(x) = f_0 + r\,\nabla f_0 + \frac{r(r-1)}{2!}\nabla^2 f_0 + \ldots\ldots + \frac{r(r-1)\ldots\ldots(r-n-1)}{n!}\nabla^n f_0$$

Newton's Forward & Backward Difference

Try to solve example on the paper

Hermite Interpolation

- Ordinary interpolation (like Lagrange or Newton) uses only function values $f(x_i)$.
- Sometimes, we also know **derivatives** $f'(x_i)$.
- **Hermite interpolation** uses both, giving a more accurate polynomial.

Hermite Interpolation

For points $x_0, x_1, \ldots, x_n$ with known values $f(x_i)$ and derivatives $f'(x_i)$,
we can construct a polynomial of degree $\leq 2n + 1$ such that:

$$P(x_i) = f(x_i), \quad P'(x_i) = f'(x_i) \quad \text{for } i = 0, 1, \ldots, n$$

This polynomial is called the **Hermite interpolant**.

Hermite Interpolation

For $n+1$ data points, Hermite interpolation constructs a polynomial of degree up to $2n+1$.

The polynomial is given by:

$$P_{2n+1}(x) = \sum_{i=0}^{n} A_i(x) f_i + \sum_{i=0}^{n} B_i(x) f_i'$$

where:

- $f_i = f(x_i)$
- $f_i' = f'(x_i)$

Definitions of Coefficients

$$A_i(x) = \left[1 - 2(x - x_i)L_i'(x_i)\right] L_i^2(x)$$

$$B_i(x) = (x - x_i)L_i^2(x)$$

Here, $L_i(x)$ are the **Lagrange basis polynomials** defined as:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

Hermite Interpolation

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

Example

x	-1	0	1
$f(x)$	1	1	3
$f'(x)$	-5	1	7

For points $x_0 = -1, x_1 = 0, x_2 = 1$:

$$L_0(x) = \frac{(x-0)(x-1)}{(-1-0)(-1-1)} = \tfrac{1}{2}(x^2 - x)$$

$$L_1(x) = \frac{(x+1)(x-1)}{(0+1)(0-1)} = -x^2 + 1$$

$$L_2(x) = \frac{(x+1)(x-0)}{(1+1)(1-0)} = \tfrac{1}{2}(x^2 + x)$$

Derivatives:

$$L_0'(x) = \tfrac{1}{2}(2x - 1), \quad L_1'(x) = -2x, \quad L_2'(x) = \tfrac{1}{2}(2x + 1)$$

Hermite Interpolation

$$A_i(x) = [1 - 2(x - x_i)L_i'(x_i)]\, L_i^2(x)$$

$$B_i(x) = (x - x_i)L_i^2(x)$$

Using formulas:

For $i = 0$:

$$A_0(x) = [1 - 2(x + 1)L_0'(-1)]\, L_0^2(x) = \tfrac{1}{4}\left(3x^5 - 2x^4 - 5x^3 + 4x^2\right)$$

For $i = 1$:

$$A_1(x) = [1 - 2(x - 0)L_1'(0)]\, L_1^2(x) = x^4 - 2x^2 + 1$$

For $i = 2$:

$$A_2(x) = [1 - 2(x - 1)L_2'(1)]\, L_2^2(x) = \tfrac{1}{4}\left(-3x^5 + 2x^4 + 5x^3 - 4x^2\right)$$

$B_i(x)$:

$$B_0(x) = (x + 1)L_0^2(x) = \tfrac{1}{4}(x^5 - x^4 - x^3 + x^2)$$

$$B_1(x) = (x - 0)L_1^2(x) = x^5 - 2x^3 + x$$

$$B_2(x) = (x - 1)L_2^2(x) = \tfrac{1}{4}(x^5 + x^4 - x^3 - x^2)$$

Hermite Interpolation

$$P_{2n+1}(x) = \sum_{i=0}^{n} A_i(x) f_i + \sum_{i=0}^{n} B_i(x) f_i'$$

Substitute into the Hermite formula:

$$P_5(x) = A_0(x)f_0 + A_1(x)f_1 + A_2(x)f_2 + B_0(x)f_0' + B_1(x)f_1' + B_2(x)f_2'$$

Substitute given values $f_0 = 1, f_1 = 1, f_2 = 3, f_0' = -5, f_1' = 1, f_2' = 7$:

$$P_5(x) = 2x^4 - x^2 + x + 1$$

$P_5(x) = 2x^4 - x^2 + x + 1$

Hermite Interpolation

$$f(-0.5) \approx P_5(-0.5) = 2(-0.5)^4 - (-0.5)^2 + (-0.5) + 1 = \frac{3}{8}$$

$$f(0.5) \approx P_5(0.5) = 2(0.5)^4 - (0.5)^2 + 0.5 + 1 = \frac{11}{8}$$

Runge's Phenomenon

- Runge's Phenomenon occurs in polynomial interpolation, when we try to approximate a function with a single high-degree polynomial over an interval using equally spaced interpolation points.
- Instead of getting a better approximation as we increase the number of interpolation points, **the opposite happens**.
- The polynomial starts to oscillate strongly near the edges (endpoints) of the interval.
- The approximation error becomes very large at the boundaries, even though it may still look good in the middle.

Runge's Phenomenon

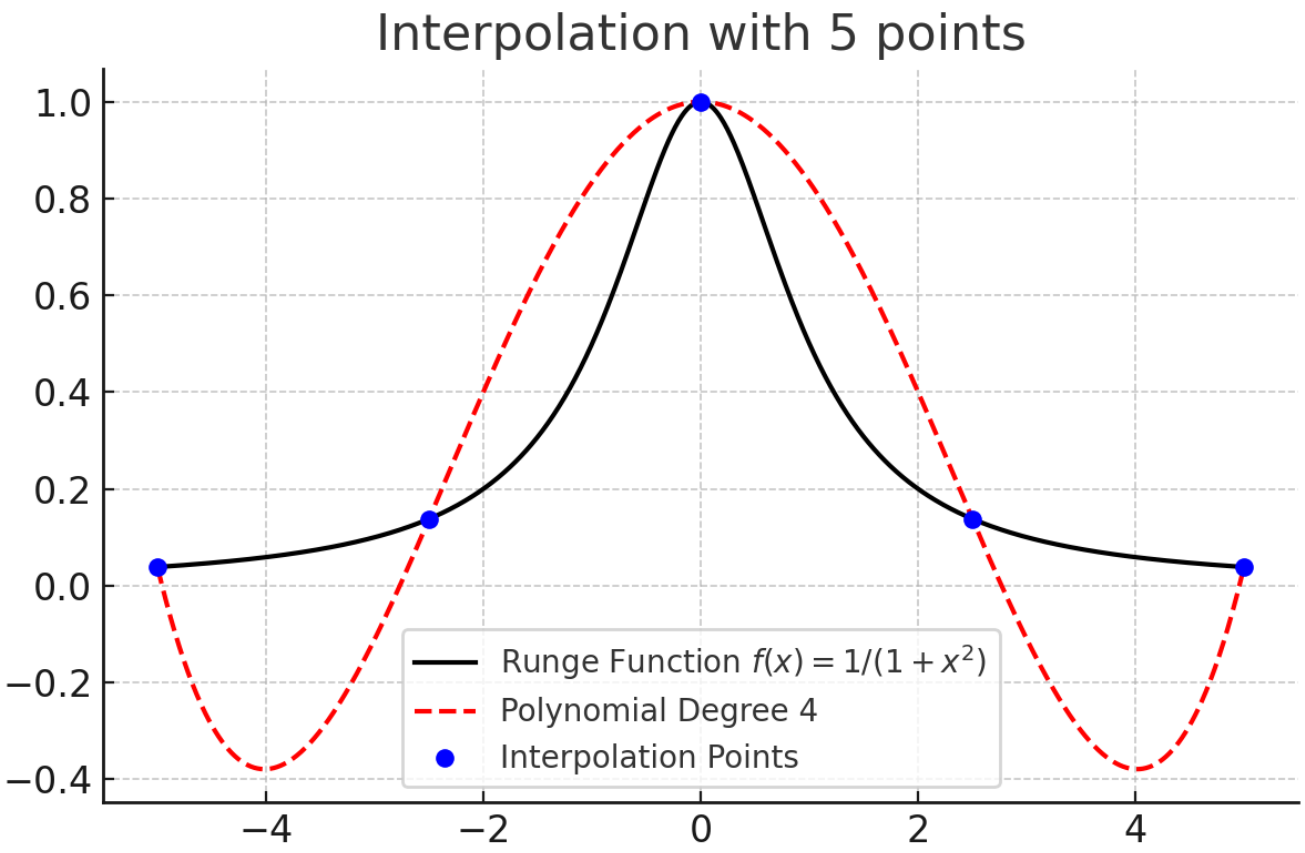
It was first observed by the German mathematician **Carl Runge** in 1901.

He demonstrated it with the function:

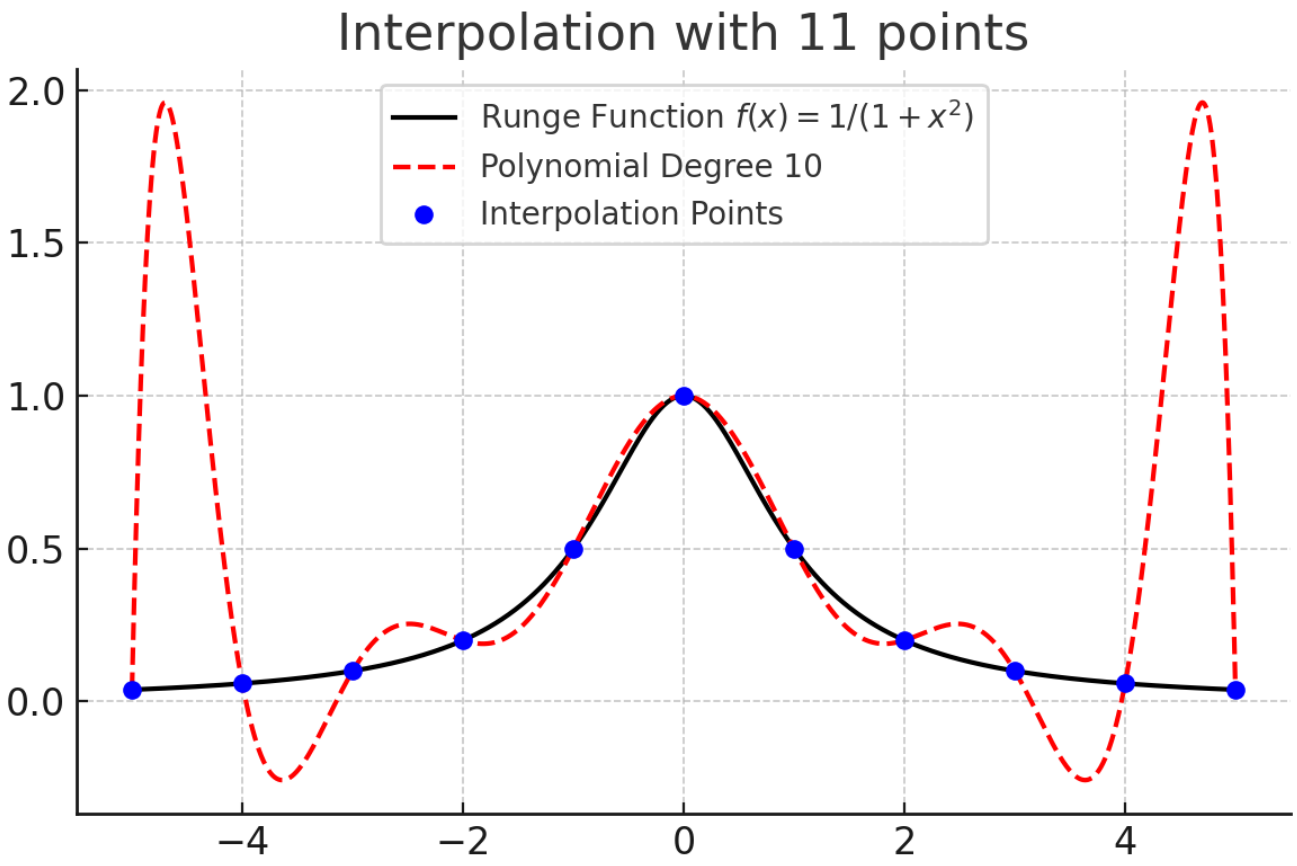
$$f(x) = \frac{1}{1+x^2}, -5 \leq x \leq 5$$

When interpolated with equally spaced points using higher and higher-degree polynomials, the approximation gets worse near $x = -5$and $x = 5$.

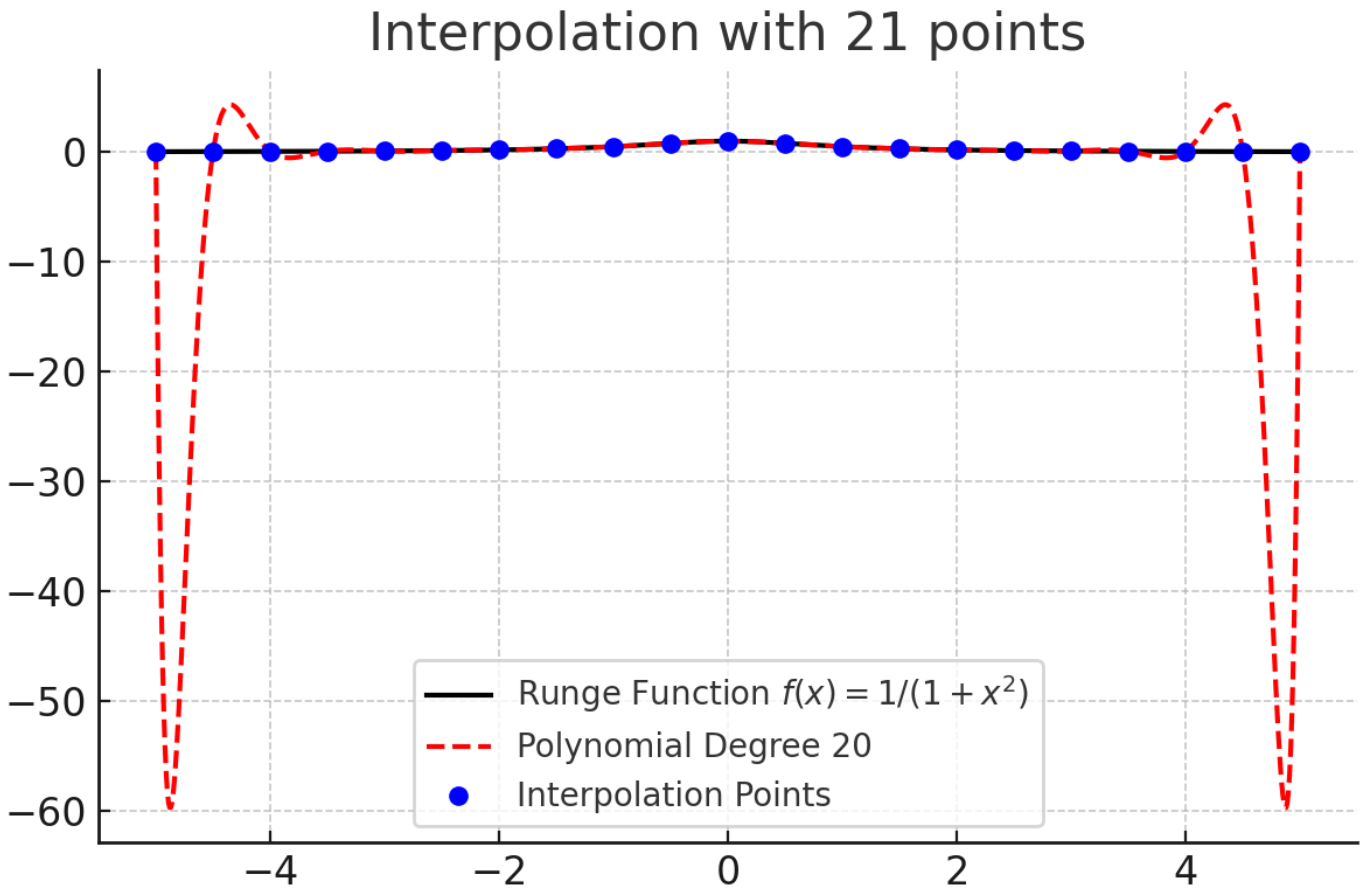
Runge's Phenomenon



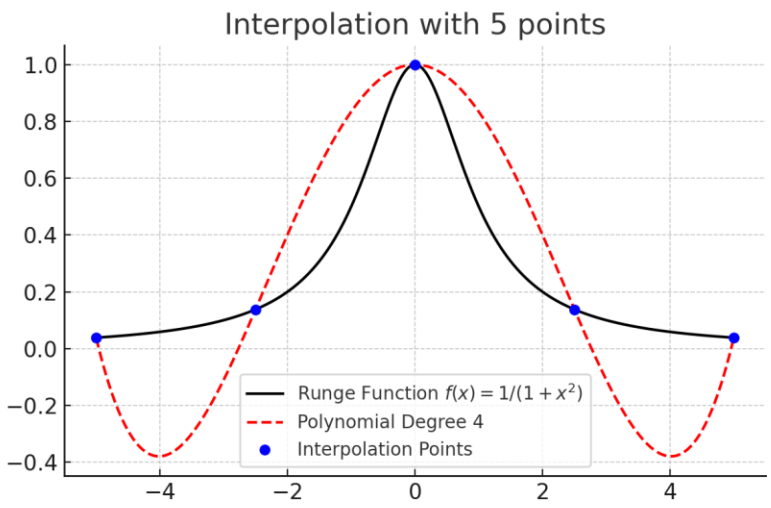
Runge's Phenomenon

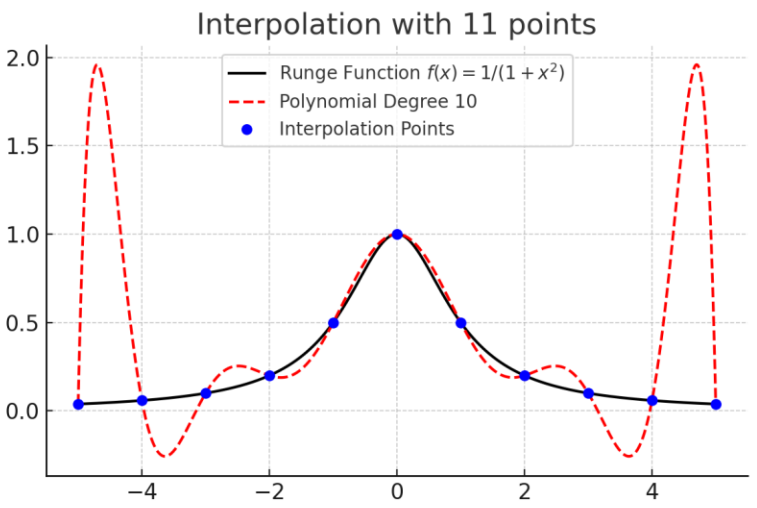


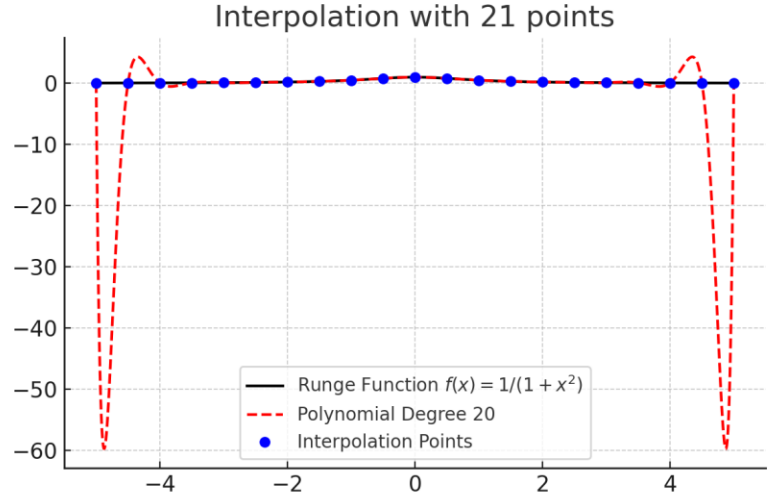
Runge's Phenomenon



Runge's Phenomenon







Runge's Phenomenon

- High-degree polynomials are unstable they tend to oscillate strongly.
- Equally spaced points amplify the oscillations near the interval boundaries.

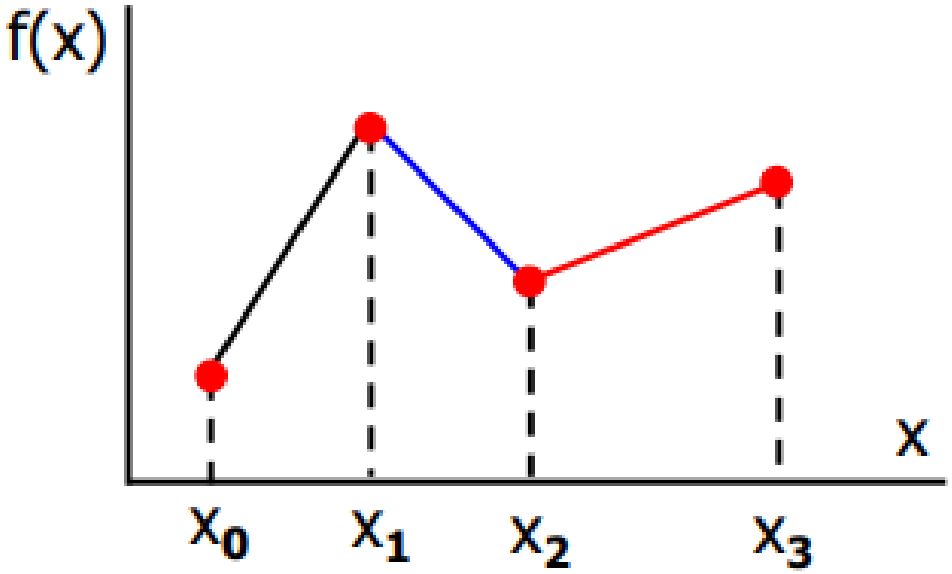
Solution:

- Piecewise interpolation (Splines)
- Instead of one big polynomial, use many low-degree polynomials joined smoothly
- Least Squares Approximation

Spline Interpolation

Given $(x_0,y_0),(x_1,y_1),\ldots,(x_{n-1},y_{n-1}),(x_n,y_n)$, fit linear splines to the data. This simply involves forming the consecutive data through straight lines.

Linear Spline Interpolation



Spline Interpolation

Linear Splines:

- Given a set of ordered data points, each two point can be connected using a straight line.

$$f(x) = f(x_0) + m_0(x - x_0) \quad \text{for } x_0 \leq x \leq x_1$$

$$f(x) = f(x_1) + m_1(x - x_1) \quad \text{for } x_1 \leq x \leq x_2$$

$$f(x) = f(x_2) + m_2(x - x_2) \quad \text{for } x_2 \leq x \leq x_3$$

where the slopes are

$$m_i = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

Spline Interpolation

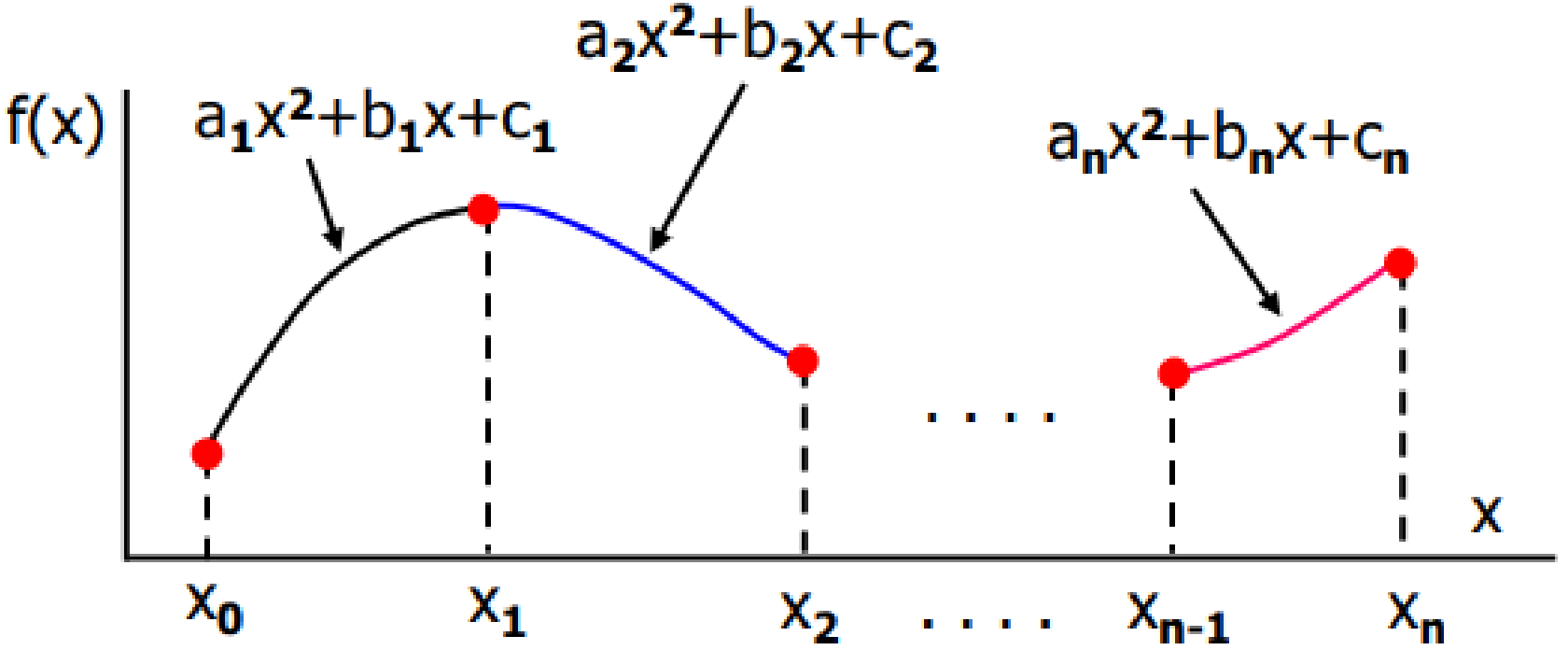
$$
\begin{aligned}
f(x) &= f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0), && x_0 \le x \le x_1, \\
&= f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1), && x_1 \le x \le x_2, \\
&\vdots \\
&= f(x_{n-1}) + \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}(x - x_{n-1}), && x_{n-1} \le x \le x_n.
\end{aligned}
$$

Note the terms of

$$\frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$$

in the above function are simply slopes between x_{i-1} and x_i.

Quadratic Spline Interpolation



Quadratic Spline Interpolation

Given ordered data $(x_0, y_0), \ldots, (x_n, y_n)$, we place a quadratic on each interval $[x_{i-1}, x_i]$ for $i = 1, \ldots, n$:

$$s_i(x) = a_i x^2 + b_i x + c_i.$$

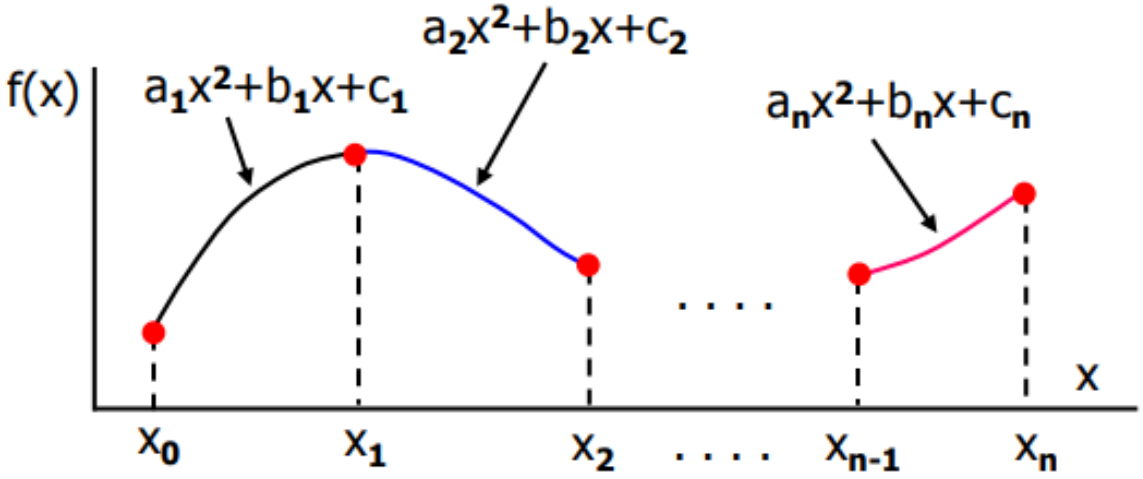
There are n intervals and $3n$ **unknowns** $\{a_i, b_i, c_i\}$. So we need $3n$ **equations** to determine them.

Quadratic Spline Interpolation

The first and last functions must pass through the end points (2 equations).

$$a_1 x_0^2 + b_1 x_0 + c_1 = f(x_0)$$

$$a_n x_n^2 + b_n x_n + c_n = f(x_n)$$

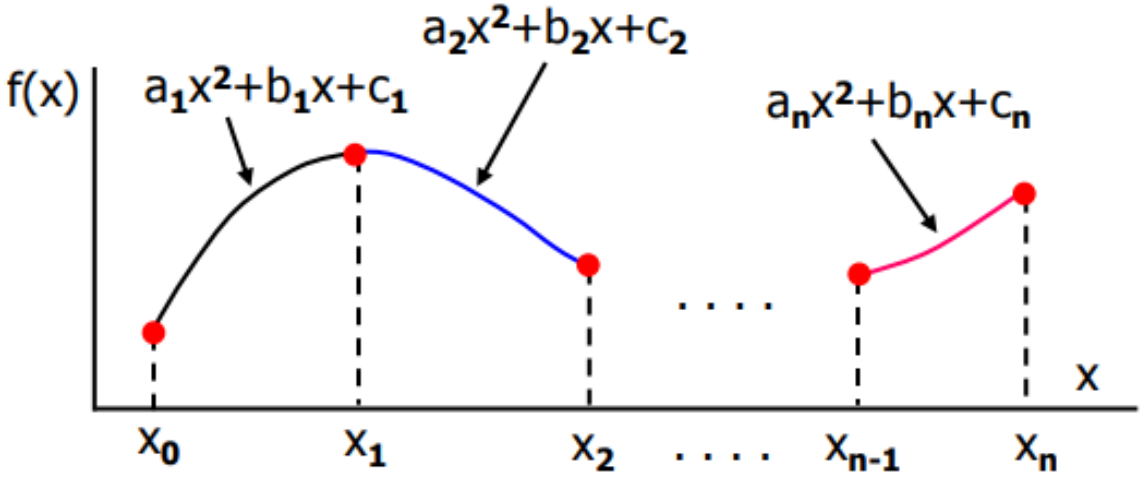


Quadratic Spline Interpolation

The function values must be equal at interior points (2n-2 equations).

At every interior point x_{i-1} for $i = 2, \ldots, n$:

$$a_{i-1}x_{i-1}^2 + b_{i-1}x_{i-1} + c_{i-1} = f(x_{i-1}),$$
$$a_i x_{i-1}^2 + b_i x_{i-1} + c_i = f(x_{i-1}).$$



Quadratic Spline Interpolation

First derivatives must be equal at the interior points (n-1 equations).

Match the slopes at each interior x_{i-1}:

$$2a_{i-1}x_{i-1} + b_{i-1} = 2a_i x_{i-1} + b_i.$$

Example

x	1	2	2.5	3	4
$f(x)$	1	5	7	8	2

Quadratic Spline Interpolation

x	1	2	2.5	3	4
$f(x)$	1	5	7	8	2

- There are 5 points and n=4 splines. Totally there are 3n=12 unknowns. Equations are
- End points: $a_1 1^2 + b_1 1 + c_1 = 1$, $a_4 4^2 + b_4 4 + c_4 = 2$
- Interior points: $a_1 2^2 + b_1 2 + c_1 = 5$, $a_2 2^2 + b_2 2 + c_2 = 5$

  $a_2 2.5^2 + b_2 2.5 + c_2 = 7$, $a_3 2.5^2 + b_3 2.5 + c_3 = 7$

  $a_3 3^2 + b_3 3 + c_3 = 8$, $a_4 3^2 + b_4 3 + c_4 = 8$
- Derivatives at the interior points: $2a_1 2 + b_1 = 2a_2 2 + b_2$

  $2a_2 2.5 + b_2 = 2a_3 2.5 + b_3$

  $2a_3 3 + b_3 = 2a_4 3 + b_4$
- Arbitrary choice for the missing equation: $a_1 = 0$

Quadratic Spline Interpolation

Quadratic spline

For n+1 points, there will be n intervals and for each interval there will be a 2nd order polynomial

Total unknowns: 3n. Use the following equations:

- The first and last functions must pass through the end points (2 equations).
- The function values must be equal at interior points (2n–2 equations).
- First derivatives must be equal at the interior points (n–1 equations).
- These give (2 + (2n-2) + (n-1) = 3n-1) equations; **one additional boundary condition is required** to reach (3n) equations.

Common choices (pick one): a1 = 0

Cubic Spline Interpolation

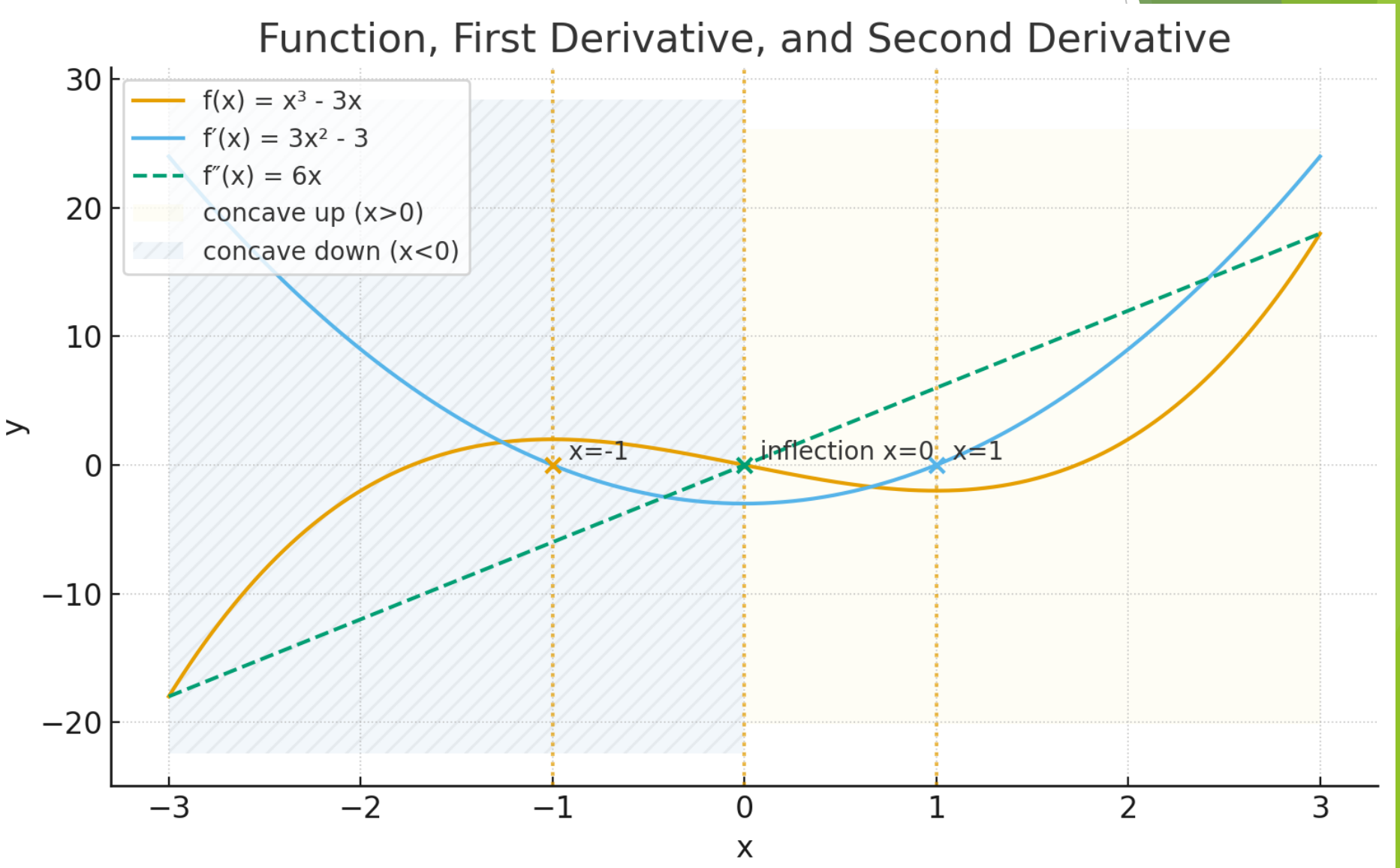
For n+1 points, there will be n intervals and for each interval there will be a 3rd order polynomial

$$a_i x_i^3 + b_i x_i^2 + c_i x + d_i \qquad \text{for } i = 1 \text{ to } n$$

There are 4n unknowns in total. They can be solved using the following equations

- The first and last functions must pass through the end points (2 equations).
- The function values must be equal at interior points (2n-2 equations).
- First derivatives must be equal at the interior points (n-1 equations).
- Second derivatives must be equal at the interior points (n-1 equations).
- This makes a total of 4n-2 equations. Two extra equations are (other choices are possible)
- Second derivatives at the end points are zero (2 equations).

Setting up and solving 4n equations is costly. There is another way of constructing cubic splines that results in only n-1 equations in n-1 unknowns.


Function, First Derivative, and Second Derivative
f(x) = x³ - 3x
f′(x) = 3x² - 3
f″(x) = 6x
concave up (x>0)
concave down (x<0)
x=-1
inflection x=0
x=1
30
20
10
0
−10
−20
−3
−2
−1
0
1
2
3
x
y

Clamped Cubic Spline Interpolation

The Problem with the Standard Method

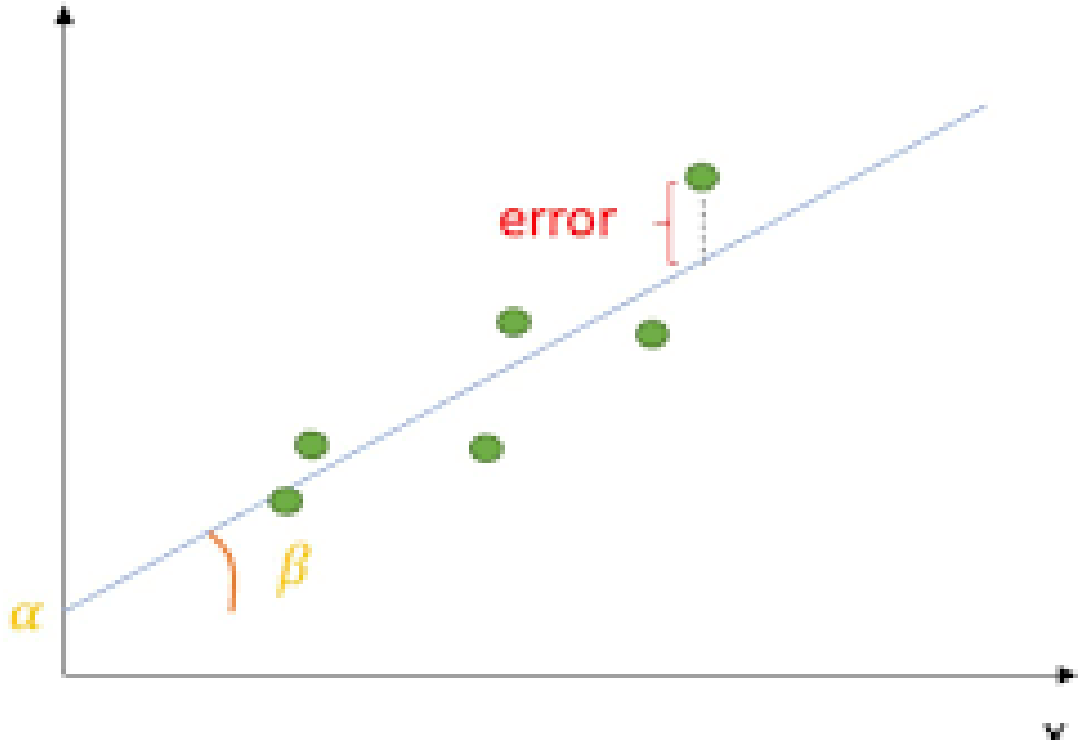
Standard cubic splines for n+1 points involve solving for 4n coefficients.
This system requires 4n-2 equations and two additional boundary conditions.

The Efficient Alternative Method

Solve for the second derivatives of the spline at the interior points, rather than all polynomial coefficients.
The system is reduced to only n-1 unknowns
A linear system of n-1 equations is formed by enforcing continuity of the first derivative at each interior point.

Least Square Method

Least Square Method



Least Square Method

Given a set of data points

x	x_0	x_1	x_2	$\cdots$	x_N
y	y_0	y_1	y_2	$\cdots$	y_N

we will compute a strait line

$$y = ax + b$$

that somehow is very close to these points.

If the kth point (x_k, y_k) is on the line, then

$$ax_k + b - y_k = 0$$

but if doesn't, then the error will be

$$|ax_k + b - yk|$$

Least Square Method

The total absolute error for all N+1 points will be

$$\sum_{k=0}^{N} |ax_k + b - y_k|$$

We then want to find the values of the coefficients a and b such that minimizes such an error. Because the absolute value of a function results to a non-smooth (non-differentiable) function we prefer to work with the square of the errors

$$S(a,b) = \sum_{k=0}^{N} (ax_k + b - y_k)^2$$

Least Square Method

where its extrimum is at the point where the partial derivative

$$\frac{\partial}{\partial a} S(a,b) = 0 \quad \text{and} \quad \frac{\partial}{\partial b} S(a,b) = 0$$

These equations lead to the following two equations

$$\sum_{k=0}^{N} 2(ax_k + b - y_k)x_k = 0$$

and

$$\sum_{k=0}^{N} 2(ax_k + b - y_k) = 0$$

Least Square Method

Rearanging these equations we get the so called **normal equations**:

$$\left(\sum_{k=0}^{N} x_k^2\right) a + \left(\sum_{k=0}^{N} x_k\right) b = \sum_{k=0}^{N} y_k x_k$$

$$\left(\sum_{k=0}^{N} x_k\right) a + \quad (N+1) \; b = \sum_{k=0}^{N} y_k$$

where we used the fact that $\sum_{k=0}^{N} 1 = N + 1$

Least Square Method

The last is a 2×2 system of linear equations with unknowns the coefficients a and b.

Denoting

$$p = \sum_{k=0}^{N} x_k, \quad q = \sum_{k=0}^{N} y_k, \quad r = \sum_{k=0}^{N} y_k x_k, \quad s = \sum_{k=0}^{N} x_k^2$$

it can be written in matrix-vector form as:

$$\begin{bmatrix} s & p \\ p & N+1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ q \end{bmatrix}$$

Cramer's Rule

Consider a system of n linear equations:

$$\begin{cases} a_1x + b_1y + c_1z = d_1 \\ a_2x + b_2y + c_2z = d_2 \\ a_3x + b_3y + c_3z = d_3 \end{cases}$$

or in matrix form:

$$A\mathbf{x} = \mathbf{b}$$

where

$$A = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \end{bmatrix}$$

Cramer's Rule

Let $D = \det(A)$.

If $D \neq 0$, then the system has a **unique solution**.

Now create three new matrices by **replacing one column** of A at a time with $\mathbf{b}$:

- D_x: Replace 1st column of A with $\mathbf{b}$
- D_y: Replace 2nd column of A with $\mathbf{b}$
- D_z: Replace 3rd column of A with $\mathbf{b}$

$$x = \frac{D_x}{D}, \quad y = \frac{D_y}{D}, \quad z = \frac{D_z}{D}$$

Least Square Method

Using Cramer's rule to solve it we need to compute first the determinant

$$D = \det \begin{bmatrix} s & p \\ p & N+1 \end{bmatrix} = (N+1)s - p^2$$

and then the solution is:

$$a = \frac{1}{D} \det \begin{bmatrix} r & p \\ q & N+1 \end{bmatrix} = [(N+1)r - pq]/D$$

and

$$b = \frac{1}{D} \det \begin{bmatrix} s & r \\ p & q \end{bmatrix} = [sq - pr]/D$$

Least Square Method

Given the data points (x_k, y_k) for $k = 0, \ldots, N$, we can express the previous process in a very simple algorithm:

Algorithm

Compute the following:

1. $p = \sum_{k=0}^{N} x_k$
2. $q = \sum_{k=0}^{N} y_k$
3. $r = \sum_{k=0}^{N} y_k x_k$
4. $s = \sum_{k=0}^{N} x_k^2$
5. $D = (N+1)s - p^2$
6. $a = [(N+1)r - pq]/D$
7. $b = [sq - pr]/D$